03. Comparative Analysis: Existing Platforms vs. PrismSpace

Architectural Benchmark: Evaluating PrismSpace against industry-standard conversational AI, autonomous agents, and swarm frameworks.

Capability Dimension	ChatGPT Web (OpenAI)	AutoGPT (Autonomous Loop)	CrewAI (Multi-Agent Swarm)	PrismSpace (Ours) Local Multi-Agent Platform
Execution Topology	Linear single-thread prompt dialog	Unbounded recursive loop (risk of infinite loops)	Sequential or static hierarchical agent chains	**Dynamic DAG Scheduling:** Parallel execution ($K \leq 5$, join barriers)
Model Strategy & Routing	Vendor locked (single OpenAI model per session)	Single model API backend (typically GPT-4)	Statically configured per agent persona	**Dynamic Multi-Model Router:** Routes Groq, OpenAI, NIM by latency/cost
Tool Connectivity Protocol	Proprietary OpenAI Plugins & Function Calls	Custom Python scripts & local shell commands	LangChain tool wrappers (rigid contracts)	**Model Context Protocol (MCP):** Standardized servers (Gmail, Drive, FS, Web)
Observability & Streaming	Text token stream only (internal steps opaque)	Terminal CLI logs only; zero visual graph	Terminal text traces; server-side execution	**Live SSE Stream + Visual DAG:** Real-time node state and telemetry
Safety & Quality Control	Closed cloud content filter; no user approval gate	Fixed step budgets; no output verification	Agent self-critique (incurs token overhead)	**HITL Approval Gates + ORPO:** High-risk tasks paused; preference aligned
Persistence & Privacy	Remote cloud storage (privacy & latency exposure)	Local filesystem artifacts; no structured database	Cloud DB or ephemeral in-memory state	**Local-First PrismDB:** Zero-latency browser IndexedDB (`useLiveQuery()`)

While ChatGPT is optimized for single-model dialog and frameworks like CrewAI/AutoGPT for Python-scripted execution, **PrismSpace** uniquely unifies **dynamic DAG planning**, **open MCP tools**, **multi-model routing**, and **local-first privacy** in a reactive workspace.